

Practical 6

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Create the Server Program

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/stat.h>

#include <sys/types.h>

#define SERVER_FIFO "/tmp/server_fifo"

#define MAX_MSG 256

typedef struct {
    pid_t pid;
    char message[MAX_MSG];
} ClientMessage;

int main() {
    int fd;

    ClientMessage request;

    char client_fifo[100];
```

```
char response[MAX_MSG];

// Create the server FIFO
if (mkfifo(SERVER_FIFO, 0666) == -1) {
    printf("Server FIFO may already exist.\n");
}

printf("=====\n");
printf("SERVER STARTED\n");
printf("Waiting for client messages...\n");
printf("=====\n");

// Open FIFO for reading
fd = open(SERVER_FIFO, O_RDONLY);

if (fd == -1) {
    perror("open");
    exit(1);
}

while (1) {

    int bytes = read(fd, &request, sizeof(request));

    if (bytes > 0) {

        printf("\nClient PID: %d\n", request.pid);
        printf("Message: %s\n", request.message);
    }
}
```

```

// Create client FIFO name
sprintf(client_fifo, "/tmp/client_%d_fifo", request.pid);

// Process the message
sprintf(response,
    "Server received your message: %s",
    request.message);

// Open client's FIFO and send response
int client_fd = open(client_fifo, O_WRONLY);

if (client_fd != -1) {

    write(client_fd, response, strlen(response) + 1);

    close(client_fd);

    printf("Response sent to Client %d\n",
        request.pid);
}
}

close(fd);

return 0;

```

```
}
```

Create Client Program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <sys/stat.h>
```

```
#include <sys/types.h>
```

```
#define SERVER_FIFO "/tmp/server_fifo"
```

```
#define MAX_MSG 256
```

```
typedef struct {
```

```
    pid_t pid;
```

```
    char message[MAX_MSG];
```

```
} ClientMessage;
```

```
int main() {
```

```
    int server_fd, client_fd;
```

```
    ClientMessage request;
```

```
    char client_fifo[100];
```

```
    char response[MAX_MSG];
```

```
    // Get current process ID
```

```
pid_t pid = getpid();

// Create unique FIFO for this client
sprintf(client_fifo,
        "/tmp/client_%d_fifo",
        pid);

// Create client FIFO
mkfifo(client_fifo, 0666);

// Open server FIFO
server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1) {
    perror("Server is not running");
    exit(1);
}

request.pid = pid;

printf("Enter message: ");

fgets(request.message,
        MAX_MSG,
        stdin);

// Send request to server
write(server_fd,
```

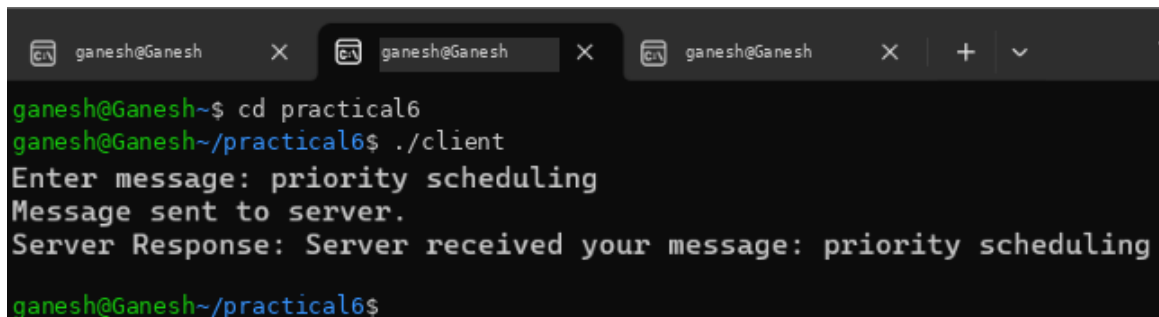
```
&request,  
sizeof(request));  
  
printf("Message sent to server.\n");  
  
close(server_fd);  
  
// Open own FIFO to receive response  
client_fd = open(client_fifo,  
O_RDONLY);  
  
if (client_fd == -1) {  
    perror("open client FIFO");  
    exit(1);  
}  
  
// Receive response  
read(client_fd,  
    response,  
    MAX_MSG);  
  
printf("Server Response: %s\n",  
    response);  
  
close(client_fd);  
// Remove client FIFO  
unlink(client_fifo);
```

```
return 0;  
}
```

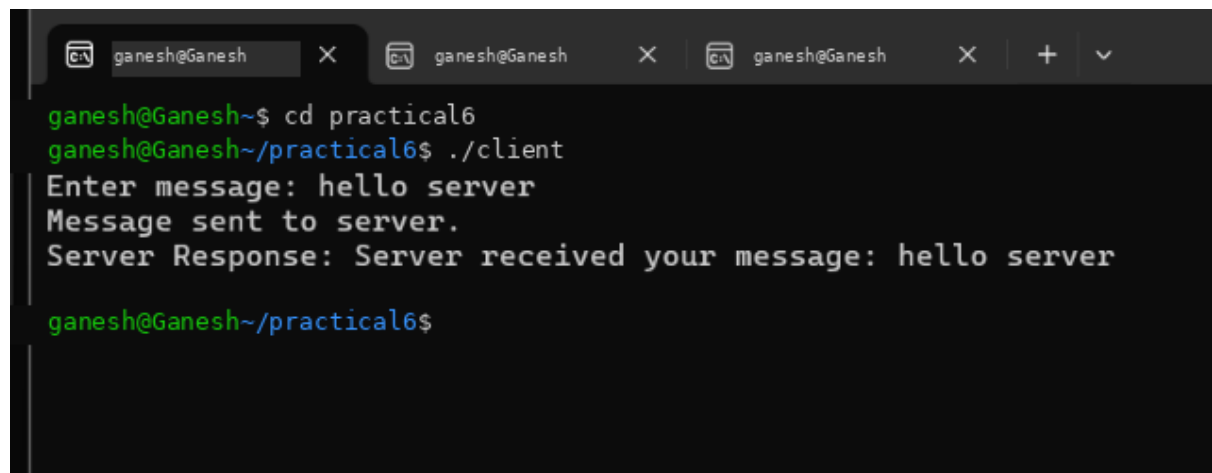
```
ganesh@Ganesh~/practical6$ nano client.c  
ganesh@Ganesh~/practical6$ gcc client.c -o client  
ganesh@Ganesh~/practical6$ cd practical6  
-bash: cd: practical6: No such file or directory  
ganesh@Ganesh~/practical6$ ./server  
=====  
SERVER STARTED  
Waiting for client messages...  
=====
```



```
Client PID: 874  
Message: hello server  
  
Response sent to Client 874  
  
Client PID: 923  
Message: priority scheduling  
  
Response sent to Client 923  
|
```

A terminal window with three tabs, all labeled 'ganesh@Ganesh'. The active tab shows the following commands and output:

```
ganesh@Ganesh~$ cd practical6  
ganesh@Ganesh~/practical6$ ./client  
Enter message: priority scheduling  
Message sent to server.  
Server Response: Server received your message: priority scheduling  
ganesh@Ganesh~/practical6$
```



A terminal window with three tabs, each labeled 'ganesh@Ganesh'. The terminal shows the following sequence of commands and output:

```
ganesh@Ganesh~$ cd practical6
ganesh@Ganesh~/practical6$ ./client
Enter message: hello server
Message sent to server.
Server Response: Server received your message: hello server
ganesh@Ganesh~/practical6$
```